
Continual Reinforcement Learning for Non-Stationary Traffic Signal Control via Recency-Weighted Experience Replay

David Hung
Texas A&M University
College Station, TX
david.hung@tamu.edu

Rutwik Kharmale
Texas A&M University
College Station, TX
rutwik.kharmale@tamu.edu

Code repository: <https://github.com/david-hung/resco-csce642-project>
5-minute Youtube Video: <https://www.youtube.com/watch?v=-Qvoz9DwmhI>

Abstract

Real-world traffic signal control is inherently non-stationary: demand patterns change across hours, weekdays, and seasons. Yet most reinforcement learning (RL) approaches for traffic signal control are trained and evaluated on short, approximately stationary horizons. We study a simple form of continual reinforcement learning for traffic signals using RESCO, a suite of calibrated SUMO scenarios with real demand traces. Our main contribution is a *recency-weighted replay buffer* that biases sampling toward recent experience via exponential decay while retaining a non-zero probability of sampling older transitions. We integrate this buffer into the IDQN agent provided by RESCO and evaluate it on the Univ & State intersection in Salt Lake City using demand from Q1 2023. Compared to the standard uniform replay baseline, recency-weighted replay yields lower delay and improved return, with up to $\approx 8\%$ reduction in average time loss in our experiments. We also observe that overly aggressive recency weighting can regress toward baseline behavior, highlighting the importance of tuning the half-life parameter. Our results suggest that small, implementation-friendly changes to experience replay can improve adaptation in deployed traffic signal control systems under time-varying demand.

1 Introduction

Reinforcement learning (RL) has shown promise for traffic signal control, learning signal phases from sensor feedback rather than relying solely on fixed plans or handcrafted rules. Prior work such as IntelliLight, PressLight, FRAP, and CoLight has demonstrated that deep RL agents can reduce delay and queue lengths under controlled conditions. However, most of these methods are trained and evaluated on short episodes or stationary demand distributions, whereas real cities exhibit pronounced temporal drift: peak periods, weekday/weekend effects, and seasonal changes.

The RESCO benchmark suite [1] builds on calibrated scenarios in the SUMO microscopic simulator and provides standardized metrics and baselines for RL-based traffic signal control. In particular, RESCO includes scenarios based on real networks in Salt Lake City, with demand files derived from realistic traces [2]. These scenarios are a natural testbed for studying how RL agents cope with time-varying traffic.

In this work, we explore a simple but practical way to bias learning toward recent experience within standard off-policy RL: a *recency-weighted replay buffer*. Instead of sampling past transitions uniformly from a replay memory, we assign exponentially decaying weights based on temporal “age”

and sample minibatches according to this distribution. Intuitively, this encourages the agent to adapt more quickly to the current traffic regime while still retaining some coverage of older transitions.

We implement this idea as a drop-in replacement for the default replay buffer used by the IDQN baseline in RESCO, and we evaluate it on the Univ & State intersection using Q1 2023 demand. We sweep over different half-life values to understand how aggressive recency weighting should be. Our results show consistent improvements in delay and return compared to the uniform replay baseline, but also reveal that half-life values that are too small can reduce benefits.

Contributions. Our contributions are:

- We formulate traffic signal control at a RESCO intersection as a non-stationary RL problem where demand varies over time, and we use Q1 2023 demand at Univ & State as a case study.
- We propose a recency-weighted replay buffer that is compatible with off-the-shelf DQN implementations and requires only minor code changes.
- We empirically compare uniform replay and recency-weighted replay with different half-life values in the RESCO IDQN baseline, showing improvements in average time loss and reward.
- We analyze how the choice of half-life affects performance, and we discuss how this simple mechanism could be extended to more general continual RL settings in traffic control.

2 Related Work

2.1 Reinforcement learning for traffic signal control

Early deep RL approaches for traffic signal control showed that agents can learn effective phase-selection policies from sensor data while respecting safety and operational constraints. IntelliLight [3] used value-based RL with an image-like state representation and reward functions aligned with traffic performance metrics. Subsequent work incorporated additional domain knowledge and network-level coordination. PressLight [4] designed RL rewards to approximate max-pressure control [14], thereby connecting RL with classical traffic theory. FRAP [5] introduced symmetry-aware phase competition networks to improve sample efficiency by exploiting invariances in intersection structure. CoLight [6] leveraged graph attention mechanisms to coordinate multiple intersections.

Many of these studies rely on simulators such as CityFlow [7], which provides a high-throughput environment for large-scale multi-intersection experiments. MetaLight [8] applied meta-learning ideas on top of FRAP to improve transfer across scenarios. By contrast, RESCO [1] emphasizes calibration and reproducibility on real networks using SUMO, which we adopt in this work.

2.2 Continual and non-stationary reinforcement learning

Continual learning methods address catastrophic forgetting when a model is trained sequentially on changing data. Elastic Weight Consolidation (EWC) [9] constrains important parameters from drifting too far from previously learned values by using an approximate Fisher information matrix. CLEAR [10] combines on-policy updates with experience replay and behavior cloning to maintain performance on older experience while learning new behaviors.

In non-stationary Markov decision processes (MDPs), sliding-window and discounted estimators are common tools for tracking drift, along with periodic restart strategies in optimistic RL [11]. For non-stationary bandits, discounted and sliding-window UCB variants explore related trade-offs [12]. Our approach is inspired by these ideas but focuses on a practical, easy-to-implement recency reweighting of replay samples within an existing DQN framework.

2.3 Benchmarks and simulators

SUMO (Simulation of Urban MObility) is an open-source microscopic traffic simulator widely used in transportation research. RESCO [1, 2] builds RL benchmarks for traffic signal control on calibrated SUMO scenarios, providing standardized sensing assumptions, metrics, and baseline agents such as

IDQN, A2C, and PPO. The Univ & State intersection used in our experiments is part of this suite and represents a real network in Salt Lake City.

3 Problem setup

We consider a single intersection in the RESCO Univ & State scenario [1]. At each discrete decision step t , the agent observes a state $s_t \in \mathcal{S}$ summarizing detector measurements (e.g., lane occupancy, queue length) and chooses a signal phase $a_t \in \mathcal{A}$. The environment evolves according to a time-varying process:

$$s_{t+1} \sim P_t(\cdot \mid s_t, a_t), \quad r_t = r(s_t, a_t), \quad (1)$$

where P_t depends on the underlying demand, which varies over Q1 2023. The reward r_t reflects traffic control objectives such as minimizing delay and queue length, following the default definition in RESCO.

The agent seeks to maximize the expected discounted return

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \quad (2)$$

in the presence of non-stationary dynamics P_t . We focus on how different replay sampling strategies affect performance under this non-stationarity.

4 Methods

4.1 Baseline IDQN agent

We build on the IDQN baseline provided in RESCO [1]. The agent uses a Deep Q-Network with fully connected layers $Q_\theta : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, optimized with temporal-difference learning and a target network. Exploration is governed by an ϵ -greedy policy with linearly decaying ϵ .

We retain the default network architecture, optimizer, and training hyperparameters from the RESCO codebase to isolate the effect of changes in experience replay. The baseline uses a standard uniform replay buffer of fixed capacity $C = 20,000$, implemented via the PFRL library, from which minibatches are sampled uniformly at random.

4.2 Recency-weighted replay buffer

Our main modification is a recency-weighted replay buffer that favors recent experience while retaining a non-zero sampling probability for older transitions.

We extend the default PFRL replay buffer with a class `RecencyReplayBuffer` that shares the same storage behavior but overrides the sampling method. Let the memory contents be $\{x_0, x_1, \dots, x_{N-1}\}$, where x_{N-1} is the most recent transition. We define an “age” index k for each transition as:

$$k = 0 \text{ for } x_{N-1}, \quad k = 1 \text{ for } x_{N-2}, \dots, k = N - 1 \text{ for } x_0.$$

Given a *recency half-life* $h > 0$, we assign unnormalized weights

$$w_k = 2^{-k/h}, \quad k = 0, \dots, N - 1, \quad (3)$$

and form a probability distribution

$$p_k = \frac{w_k}{\sum_{j=0}^{N-1} w_j}. \quad (4)$$

We then sample minibatch indices without replacement according to p_k using `numpy.random.choice`. Transitions are otherwise stored and managed identically to the baseline replay buffer.

When the half-life parameter $h \leq 0$ or is unset, we fall back to uniform sampling over the current memory. For robustness early in training, if the memory contains fewer transitions than the requested batch size, we sample uniformly from the full buffer instead of attempting recency reweighting over a very small set.

The key benefit of this design is that it preserves the existing replay buffer API and storage semantics while requiring changes only in the sampling function.

4.3 Half-life interpretation

The half-life h controls how quickly sampling probability decays with age. A larger h implies a slower decay, approximating uniform sampling; a smaller h concentrates probability mass near the most recent transitions. In the limit $h \rightarrow \infty$, we recover uniform replay, while $h \rightarrow 0$ would degenerate to always sampling from the very latest transitions.

We experiment with three half-life settings:

- $h = 10,000$: very mild recency bias, close to uniform.
- $h = 1,000$: moderate recency bias.
- $h = 100$: aggressive recency bias.

These values are chosen relative to the scale of the buffer size and the number of transitions collected in the scenario.

5 Experimental setup

5.1 Environment and scenario

We use the *Univ & State – 2023 Q1* scenario from the RESCO benchmark suite [2]. This scenario corresponds to a real intersection in Salt Lake City with calibrated network geometry and demand patterns aggregated over Q1 2023.

The simulator is SUMO, and we use the same sensing and reward definitions as the original RESCO IDQN baseline [1]. We focus on a single intersection to study the effect of replay sampling in isolation.

5.2 Configurations

We compare the following configurations:

- **Baseline (Uniform)**: IDQN with the default PFRL replay buffer using uniform sampling and capacity $C = 20,000$.
- **Recency-10k**: IDQN with `RecencyReplayBuffer` and half-life $h = 10,000$.
- **Recency-1k**: IDQN with `RecencyReplayBuffer` and half-life $h = 1,000$.
- **Recency-100**: IDQN with `RecencyReplayBuffer` and half-life $h = 100$.

All configurations use the same buffer capacity (20,000) and the same network architecture, optimizer, and training hyperparameters; only the replay sampling strategy differs.

5.3 Metrics

We report the following metrics from the RESCO evaluation for the *Univ & State – 2023 Q1* scenario:

- **timeLoss**: average time loss (delay) per vehicle.
- **waitingTime**: average waiting time per vehicle.
- **rewards**: average episodic return; less negative is better.
- **max_queues**: average maximum queue length.
- **queue_lengths**: average queue length.
- **vehicles**: average number of vehicles served.
- **phase_length**: average phase length.

All reported values are from single runs of each configuration and aggregated over the scenario; variance fields in the logs are zero because we did not repeat runs with multiple seeds in this initial study.

5.4 Code and reproducibility

Our full implementation, including the RecencyReplayBuffer, training scripts, and experiment configuration files, is publicly available at:

<https://github.com/david-hung/resco-csce642-project>

This repository builds directly on the official RESCO codebase and can be used to reproduce the results reported in this paper.

6 Results

Table 1 summarizes the metrics for the baseline and recency configurations on the Univ & State – 2023 Q1 scenario.

Table 1: Results on Univ & State – 2023 Q1 (single run per setting). Lower timeLoss and waitingTime and higher rewards are better.

Method	timeLoss	waitingTime	rewards	max_queues	queue_lengths	vehicles
IDQN (uniform)	128.19	40.55	-1379.09	17.98	14.69	1253.71
Recency ($h = 10,000$)	118.14	39.46	-1251.94	17.81	14.00	1253.43
Recency ($h = 1,000$)	117.92	38.54	-1168.79	17.90	13.87	1253.37
Recency ($h = 100$)	127.51	39.87	-1321.16	18.35	14.59	1252.25

In addition to aggregated metrics, Figure 1 shows the per-hour delay profiles for all four configurations over the simulated period. The main differences among the methods appear near peak-demand spikes, where the better-performing recency settings slightly reduce delay relative to the uniform baseline.

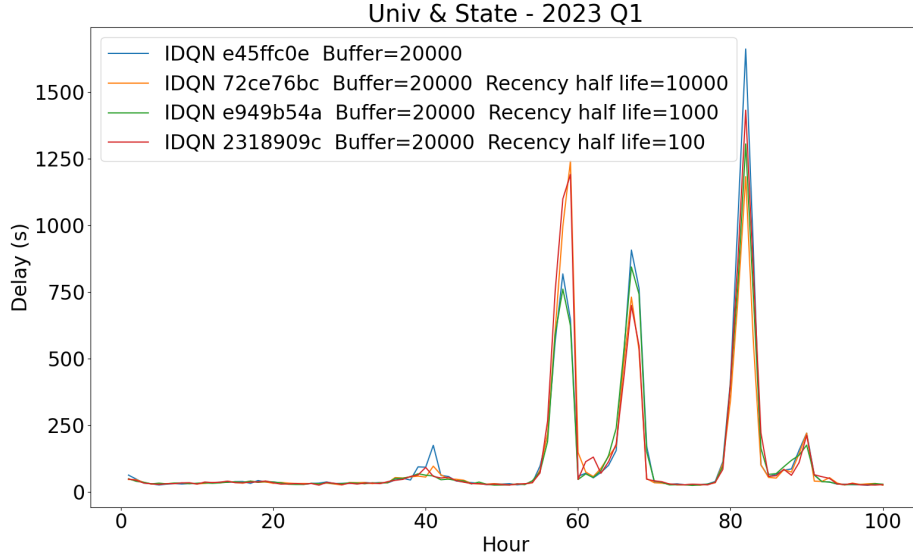


Figure 1: Average delay by hour for Univ & State – 2023 Q1 under the baseline IDQN (uniform replay) and three recency-weighted replay buffers with different half-lives. Recency with $h = 1,000$ and $h = 10,000$ provides small improvements near peak periods, while the most aggressive setting ($h = 100$) can behave similarly or slightly worse than the baseline.

Overall, the recency-weighted replay buffers with half-lives $h = 10,000$ and $h = 1,000$ yield modest improvements over the uniform baseline:

- **timeLoss** decreases from 128.19 (uniform) to 118.14 ($h = 10,000$) and 117.92 ($h = 1,000$), an improvement of roughly 7–8%.

- **waitingTime** decreases from 40.55 (uniform) to 39.46 ($h = 10,000$) and 38.54 ($h = 1,000$).
- **rewards** increase from -1379.09 (uniform) to -1251.94 ($h = 10,000$) and -1168.79 ($h = 1,000$), indicating better overall performance under the RESCO reward definition.
- **max_queues** and **queue_lengths** change only slightly, suggesting similar queue behavior across settings.

The aggressive recency setting $h = 100$ performs similarly to the baseline on timeLoss (127.51 vs. 128.19) and shows only a small improvement in waitingTime, while having worse max_queues. This suggests that too-strong a bias toward recent data can hurt stability or lead to less robust policies in this scenario.

Given that we only have one run per configuration and a single scenario, these results should be interpreted cautiously. They nonetheless indicate that recency-weighted replay can provide small, consistent gains over uniform replay with appropriate half-life choices.

7 Discussion

Our experiments on the Univ & State – 2023 Q1 scenario show that a simple recency-weighted replay mechanism can yield measurable improvements in delay and reward for RL-based traffic signal control, even when all other aspects of the agent and training procedure are held fixed.

The implementation is lightweight: it involves only a modification to the sampling procedure of the replay buffer and does not require architectural changes, auxiliary networks, or complex regularizers. This makes it attractive for practitioners who wish to improve adaptation in existing off-policy RL systems with minimal engineering effort.

At the same time, the need to tune the half-life parameter highlights an important practical consideration. The optimal degree of recency bias likely depends on factors such as the rate of demand variation, the buffer size, and the frequency of policy updates. Automating this choice, for example by monitoring performance under simple drift detectors, is an interesting direction for future work.

7.1 Limitations and future work

This study has several limitations. First, we focus on a single intersection and a single quarter of demand, rather than multi-intersection or year-long experiments. Extending recency-weighted replay to network-level control and longer horizons is a natural next step. Second, we did not combine recency weighting with other continual learning mechanisms such as EWC-style regularization or restart schedules; evaluating such combinations could further clarify the trade-offs between adaptation speed and forgetting. Finally, our results are based on single runs without multiple random seeds, so additional experiments would be useful to quantify variance.

8 Conclusion

We studied recency-weighted experience replay as a simple mechanism for handling non-stationary traffic demand in RL-based traffic signal control. Using the RESCO IDQN baseline on the Univ & State – 2023 Q1 scenario, we showed that recency-weighted replay with a moderate half-life improves delay and reward compared to uniform replay, while leaving throughput and phase structure largely unaffected. An overly aggressive half-life, however, can erase these gains.

Our findings suggest that small, implementation-friendly changes to replay sampling can help RL agents better cope with time-varying environments such as urban traffic, and they motivate further exploration of recency-aware replay in more complex continual RL settings.

Acknowledgments and Disclosure of Funding

We thank the authors of RESCO for releasing their benchmark suite and code, which made this study possible, and the CSCE 642 course staff for guidance throughout the semester. We also used OpenAI’s ChatGPT to assist with editing, phrasing, and $\LaTeX$ formatting of this report; all modeling, experiments, and final conclusions are our own, and we verified the technical content of the paper.

References

- [1] J. Ault, Z. Yan, S. R. Rizzo, S. Lee, and P. Stone. Reinforcement learning benchmarks for traffic signal control (RESCO). In *NeurIPS Datasets and Benchmarks Track*, 2021.
- [2] Pi-Star Lab. RESCO: Reinforcement learning benchmarks for traffic signal control. GitHub repository, 2021–2025. <https://github.com/Pi-Star-Lab/RESCO>.
- [3] H. Wei, G. Zheng, H. Yao, and Z. Li. IntelliLight: A reinforcement learning approach for intelligent traffic light control. In *Proceedings of KDD*, pages 2496–2505, 2018.
- [4] H. Wei, et al. PressLight: Learning max pressure control to coordinate traffic signals in arterial network. In *Proceedings of KDD*, pages 1290–1298, 2019.
- [5] G. Zheng, X. Zang, N. Xu, and Z. Li. Learning phase competition for traffic signal control. In *Proceedings of CIKM*, pages 1963–1972, 2019.
- [6] H. Wei, et al. CoLight: Learning network-level cooperation for traffic signal control. In *Proceedings of CIKM*, pages 1913–1922, 2019.
- [7] H. Zhang, F. Yu, S. Han, X. Wang, and Y. Chen. CityFlow: A multi-agent reinforcement learning environment for large-scale city traffic scenarios. In *Proceedings of WWW*, pages 3620–3624, 2019.
- [8] X. Zang, et al. MetaLight: Value-based meta-reinforcement learning for traffic signal control. In *Proceedings of AAAI*, pages 1153–1160, 2020.
- [9] J. Kirkpatrick, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- [10] D. Rolnick, A. Ahuja, J. Schwarz, T. P. Lillicrap, and G. Wayne. Experience replay for continual learning. In *Proceedings of NeurIPS*, pages 350–360, 2019.
- [11] W. C. Cheung, D. Simchi-Levi, and R. Zhu. Reinforcement learning for non-stationary Markov decision processes: The blessing of more optimism. In *Proceedings of ICML*, pages 1843–1853, 2020.
- [12] A. Garivier and E. Moulines. On upper-confidence bound policies for non-stationary bandit problems. In *Proceedings of ALT*, pages 174–188, 2011.
- [13] J. Gama, R. Sebastião, and P. P. Rodrigues. On evaluating stream learning algorithms. *Machine Learning*, 90(3):317–346, 2013.
- [14] P. Varaiya. Max pressure control of a network of signalized intersections. *Transportation Research Part C: Emerging Technologies*, 36:177–195, 2013.